Roteiro de fala e plano dos slides

Tema: Análise comparativa de desempenho entre os algoritmos de ordenação Mergesort bottom-up e Quicksort clássico
Disciplina: Estrutura de Dados · **Professor:** Me. Marcelo Goulart Souza · **Apresentação:** 13/11/2026 (Turma B)
Grupo (na ordem em que falam): Guilherme da Silva Santos · Jeferson Gomes Martins · Gustavo Grobe · Juan Flor · Igor Jesus · João Vitor Gonçalves de Almeida
Instituição: Centro Universitário UNIFACVEST · Ciência da Computação · Lages – SC

Este roteiro diz o que está em cada slide, quem fala, o que dizer e quanto tempo. Todos os números foram copiados dos resultados do artigo (os mesmos dos slides e da planilha). Ao ensaiar, **não troquem valores de memória**: se esquecer um número, aponte para a tela.

O que este plano cumpre: todos os 6 integrantes apresentam; 13:00 de apresentação, com 1 minuto de demonstração opcional, mais 2:00 para perguntas, dentro dos 15 minutos; os slides seguem a ordem do artigo (problema → método → algoritmos → resultados com estatística descritiva, boxplot e barras → conclusão).

Como foi feita a divisão: pela dicção do grupo e pela importância de cada parte. O Guilherme abre, faz a demonstração e fecha. O Jeferson explica o algoritmo do grupo, o Mergesort bottom-up. O Juan apresenta os resultados e a estatística, a parte que o professor mais cobra. O Gustavo, o Igor e o João Vitor ficam com slides mais visuais, em que o desenho ajuda a explicação e a fala é mais curta. Cada um fala em blocos seguidos, sem ir e voltar.

1. Tabela geral

Nº	Título do slide	Quem	Tempo	Acumulado
1	Mergesort × Quicksort	Guilherme	0:30	0:30
2	Como comparar algoritmos de ordenação?	Guilherme	0:45	1:15
3	Como medimos	Guilherme	0:45	2:00
—	Demonstração: o painel rodando (opcional)	Guilherme	1:00	3:00
4	Mergesort Bottom-Up: intercalação	Jeferson	1:45	4:45
5	Quicksort: dividir em torno de um pivô	Gustavo	1:15	6:00
6	Complexidade e desempenho	Gustavo	0:45	6:45
7	Resultados: tempo médio de 20 amostras	Juan	1:00	7:45
8	Estatística descritiva: 300.000 dados	Juan	1:15	9:00
9	Boxplot: distribuição das 20 amostras	Igor	1:00	10:00
10	O crescimento confirma n log n	Igor	0:45	10:45
11	Outros tipos de entrada (300.000 dados)	João Vitor	0:45	11:30
12	Vantagens e desvantagens	João Vitor	0:45	12:15
13	Qual algoritmo escolher?	Guilherme	0:45	13:00
	Perguntas	Todos (o Guilherme conduz)	2:00	15:00

Tempo por pessoa: Guilherme 3:45 (slides 1, 2, 3, 13 + demonstração) · Jeferson 1:45 (slides 4) · Gustavo 2:00 (slides 5, 6) · Juan 2:15 (slides 7, 8) · Igor 1:45 (slides 9, 10) · João Vitor 1:30 (slides 11, 12). Total: 13:00.

Se atrasar: cortem primeiro a demonstração (ganha 1:00) e encurtem o slide 6 para uma frase.

2. Slides

Slide 1 · Mergesort × Quicksort

Quem fala: Guilherme · **Tempo:** 0:30 (de 0:00 a 0:30)
Mensagem principal: Somos o grupo e vamos comparar o Mergesort bottom-up com o Quicksort clássico medindo 200 ordenações.

Na tela:
- Título: MERGESORT × QUICKSORT (azul-marinho e roxo)
- Subtítulo: Algoritmos de ordenação: funcionamento, complexidade e desempenho
- Visual: quadradinhos 7 3 9 1 6 / 4 8 2 5 → seta → 1 2 3 4 5 6 7 8 9
- Integrantes, na ordem em que falam: Guilherme da Silva Santos · Jeferson Gomes Martins · Gustavo Grobe · Juan Flor · Igor Jesus · João Vitor Gonçalves de Almeida
- Rodapé: Estrutura de Dados · Prof. Me. Marcelo Goulart Souza · Centro Universitário UNIFACVEST

FALA

Boa noite, professor e colegas. Eu sou o Guilherme, e comigo estão o Jeferson, o Gustavo, o Juan, o Igor e o João Vitor. O nosso trabalho compara dois algoritmos de ordenação: o Mergesort, na versão bottom-up, e o Quicksort clássico. Implementamos os dois em C++ e medimos 200 ordenações para responder uma pergunta simples: na prática, qual ordena mais rápido? Todos nós vamos apresentar uma parte.

Transição: *Primeiro: como comparar dois algoritmos de forma justa?*

Slide 2 · Como comparar algoritmos de ordenação?

Quem fala: Guilherme · **Tempo:** 0:45 (de 0:30 a 1:15)
Mensagem principal: Comparação justa: os mesmos dados, cinco tamanhos e 20 medições de cada caso.

Na tela:
- 5 tópicos à esquerda: o que é ordenar, o que comparamos, 20 medições, mesmos dados, conferência
- À direita: escada 30 → 300 → 3.000 → 30.000 → 300.000 elementos, com “× 20 amostras cada”

FALA

Ordenar é organizar os dados segundo uma ordem, e isso está por trás de quase tudo: busca, banco de dados, ranking. Para comparar os dois algoritmos de forma justa, usamos os cinco tamanhos do enunciado, de 30 até 300 mil números. Cada tamanho foi medido 20 vezes para cada algoritmo, o que dá 200 ordenações. E, em cada medição, os dois recebem exatamente os mesmos números, como uma prova igual para todos os alunos. No fim, conferimos se todo vetor saiu ordenado.

Transição: *E como medimos esse tempo na prática?*

Slide 3 · Como medimos

Quem fala: Guilherme · **Tempo:** 0:45 (de 1:15 a 2:00)
Mensagem principal: Medimos só a ordenação, com dados reproduzíveis, e conferimos cada resultado.

Na tela:
- 5 passos numerados: gerar (semente 42) → copiar → cronometrar → conferir → estatística e gráficos
- Cartão “Ferramentas”: C++11, Dev-C++ no Windows 11, PAINEL_ADM.bat, Python + Matplotlib, medições num servidor Linux

FALA

O programa segue cinco passos. Gera os números aleatórios com uma semente fixa, a 42, então quem rodar de novo recebe os mesmos dados. Copia esses dados para os dois algoritmos. Cronometra só a ordenação: gerar e copiar ficam fora do cronômetro. Confere se o vetor saiu ordenado. E, no fim, calcula a estatística e monta os gráficos. O código é em C++ e roda no Windows com o Dev-C++, pelo painel PAINEL_ADM.bat. Os números do artigo foram medidos num servidor Linux, e a estatística ficou numa planilha do Excel.

Transição: *Vou mostrar rapidinho o painel rodando. (Se o tempo estiver apertado, pular direto para o slide 4.)*

Demonstração · Demonstração: o painel rodando (opcional)

Quem fala: Guilherme · **Tempo:** 1:00 (de 2:00 a 3:00)
Mensagem principal: O programa roda de verdade: 200 ordenações em segundos e a tabela no formato do professor.

Na tela:
- Sem slide próprio: tirar a apresentação da tela (Alt+Tab) e mostrar o PAINEL_ADM.bat; ver o passo a passo na seção 3

FALA

Este é o painel que fizemos. Com a opção 2, ele roda as 200 ordenações em uns dois segundos e abre um relatório no navegador. Com a opção 11, mostra a estatística no formato que o professor pediu, uma tabela por tamanho. Os números desta máquina podem sair diferentes dos do slide, porque é outro computador, mas o padrão é o mesmo: empate nos vetores pequenos e o Quicksort na frente nos grandes.

Transição: *Voltando aos slides: o Jeferson explica o algoritmo do nosso grupo, o Mergesort bottom-up.*

Slide 4 · Mergesort Bottom-Up: intercalação

Quem fala: Jeferson · **Tempo:** 1:45 (de 3:00 a 4:45)
Mensagem principal: O bottom-up não divide: junta blocos de baixo para cima, dobrando o tamanho; n log n sempre, estável, com memória extra.

Na tela:

- 3 etapas numeradas à esquerda
- Diagrama com 9 números em 5 linhas: blocos de 1 → de 2 → de 4 → de 8 → pronto
- Quadro “No dia a dia”: torneio de chaves
- Faixa embaixo: Complexidade O(n log n) em qualquer caso · Estável · Usa memória auxiliar O(n)

FALA

O algoritmo do nosso grupo é o Mergesort, na versão bottom-up, de baixo para cima. A versão mais conhecida, a top-down, divide o vetor ao meio várias vezes, com recursão. A bottom-up pula essa parte: ela parte da ideia de que cada número sozinho já é um bloco ordenado. Na primeira passada, junta os blocos vizinhos de dois em dois: o 7 com o 3 vira 3 e 7, o 9 com o 1 vira 1 e 9. Na segunda, junta blocos de dois e forma blocos de quatro. Depois, de oito. Com nove números são quatro passadas, e o 5, que sobrou sozinho, espera o seu par. Juntar, ou intercalar, é comparar o primeiro de cada bloco e pegar o menor. É como um torneio de chaves: jogam as duplas, depois os quartetos, até sobrar o campeão. Como o bloco dobra a cada passada, com 300 mil números são só 19 passadas, e cada uma percorre todos os elementos. Por isso o custo é n log n em qualquer caso. Ele é estável, ou seja, mantém a ordem dos elementos iguais, mas precisa de um vetor auxiliar do mesmo tamanho, e é aí que ele perde tempo.

Transição: *Agora o Gustavo mostra o outro lado: o Quicksort.*

Slide 5 · Quicksort: dividir em torno de um pivô

Quem fala: Gustavo · **Tempo:** 1:15 (de 4:45 a 6:00)
Mensagem principal: O Quicksort escolhe o pivô do meio, separa menores e maiores no próprio vetor e repete em cada lado.

Na tela:
- Vetor 7 3 9 1 6 4 8 2 5 com o 6 destacado (“pivô = elemento do meio”)
- Linha do meio: 5 3 2 1 4 | 6 8 9 7 (menores ou iguais a 6 | maiores ou iguais a 6)
- Linha de baixo: 1 2 3 4 5 | 6 7 8 9 (repete em cada lado)
- 3 tópicos, quadro “No dia a dia” (nota de corte) e as faixas O(n log n) / O(n²) e “Ordena no próprio vetor”

FALA

O Quicksort também divide o problema, mas de outro jeito. Primeiro, escolhe um pivô. Na versão clássica que usamos, o pivô é o elemento do meio: aqui, o 6. Aí entram dois índices. Um vem da esquerda procurando quem é maior ou igual a 6, o outro vem da direita procurando quem é menor ou igual, e os dois trocam de lugar. Quando eles se cruzam, o vetor está dividido: de um lado 5, 3, 2, 1 e 4; do outro, 6, 8, 9 e 7. Depois, ele repete a mesma coisa em cada lado, até tudo ficar em ordem. É como separar provas por uma nota de corte: as menores numa pilha, as maiores na outra. A grande vantagem é que ele faz tudo no próprio vetor, sem vetor extra.

Transição: *Mas o Quicksort tem um ponto fraco, que aparece na complexidade.*

Slide 6 · Complexidade e desempenho

Quem fala: Gustavo · **Tempo:** 0:45 (de 6:00 a 6:45)
Mensagem principal: Os dois são n log n na média, mas o Quicksort pode virar n² com um pivô ruim.

Na tela:
- Gráfico de linhas (escala log): n log n × n², de 30 a 300.000
- Destaque: com 300.000 itens, n log n ≈ 5,5 milhões de passos; n² ≈ 90 bilhões
- Quadro “No dia a dia”: festa de 300 mil pessoas

FALA

Na teoria, o Mergesort é n log n no melhor, no médio e no pior caso. O Quicksort é n log n na média, mas, se o pivô for sempre o menor ou o maior elemento, ele vira n ao quadrado. O gráfico mostra o tamanho dessa diferença: com 300 mil itens, n log n dá uns 5,5 milhões de passos; n ao quadrado, 90 bilhões. É como uma festa de 300 mil pessoas em que todo mundo cumprimenta todo mundo.

Transição: *Na teoria, os dois empatam na média. E na prática? O Juan mostra os resultados.*

Slide 7 · Resultados: tempo médio de 20 amostras

Quem fala: Juan · **Tempo:** 1:00 (de 6:45 a 7:45)
Mensagem principal: Empate até 3.000 itens; o Quicksort foi 8,9% e 15,2% mais rápido com 30 mil e 300 mil.

Na tela:
- Tabela: Itens × Mergesort (ms) × Quicksort (ms) × Resultado
- Cartão roxo: 15% — Quick mais rápido com 300.000 dados
- Cartão claro: Até 3.000 dados, diferenças dentro da variação das medições

FALA

Esta tabela mostra o tempo médio das 20 amostras, em milissegundos. Até 3 mil números, os dois algoritmos empataram: as diferenças foram menores que a variação natural das medições, e com 30 e 300 itens o Mergesort chegou a ficar um pouco na frente. A partir de 30 mil, o Quicksort passa a ganhar sempre: foi 8,9% mais rápido com 30 mil e 15,2% mais rápido com 300 mil, com 23,1 milissegundos contra 27,3. Para ter uma ideia, uma piscada de olho dura uns 100 milissegundos: os dois ordenam 300 mil números em menos de um terço disso.

Transição: *Só a média não basta: o enunciado pede a estatística completa.*

Slide 8 · Estatística descritiva: 300.000 dados

Quem fala: Juan · **Tempo:** 1:15 (de 7:45 a 9:00)
Mensagem principal: A tabela segue o formato pedido; a mediana confirma a vantagem do Quicksort, com dispersão moderada.

Na tela:
- Tabela: Algoritmo × Média × Mediana × Desvio × Variância × CV (%) × Máx/Mín
- 3 cartões: 14% de mediana menor no Quicksort · CV de 11% e 12% · 5 tabelas no artigo
- Quadro “No dia a dia”: a mediana é o aluno do meio da fila

FALA

Esta é a tabela no formato que o professor pediu, para o maior conjunto, 300 mil dados. Além da média, calculamos a mediana, que é o valor do meio das 20 amostras: 24,4 milissegundos no Quicksort contra 28,4 no Mergesort, 14% a menos. O desvio padrão e a variância mostram o quanto os tempos se espalham, e o coeficiente de variação coloca isso em porcentagem da média: ficou em 10,9% e 12,5%, uma dispersão moderada. A mediana é como o aluno do meio da fila: não muda se um único aluno chegar muito atrasado. Por isso ela confirma o que a média mostrou. As tabelas dos cinco tamanhos estão no artigo e na planilha do Excel.

Transição: *O Igor mostra essas 20 amostras de outro jeito: no boxplot.*

Slide 9 · Boxplot: distribuição das 20 amostras

Quem fala: Igor · **Tempo:** 1:00 (de 9:00 a 10:00)
Mensagem principal: Caixas sobrepostas significam empate; caixas separadas, diferença real.

Na tela:

- Imagem com 5 boxplots (30, 300, 3.000, 30.000 e 300.000 dados), Merge em azul e Quick em roxo
- Faixa embaixo: Até 3.000, caixas se sobrepõem → empate · 30.000 e 300.000, Quick mais baixo → mais rápido

FALA

O boxplot mostra todas as 20 medições de cada caso. A caixa vai do primeiro ao terceiro quartil, ou seja, guarda a metade do meio das amostras, e a linha dentro dela é a mediana. Os pontinhos soltos são medições fora do padrão, quase sempre mais lentas por alguma interferência do sistema. Usamos uma regra simples: se as caixas dos dois algoritmos se sobrepõem, é empate. Até 3 mil itens, elas se sobrepõem. Com 30 mil, as caixas se separam, e a do Quicksort fica mais embaixo. Com 300 mil, a caixa do Quicksort continua mais baixa.

Transição: *E como esse tempo cresce quando os dados aumentam?*

Slide 10 · O crescimento confirma n log n

Quem fala: Igor · **Tempo:** 0:45 (de 10:00 a 10:45)
Mensagem principal: Dados ×10 → tempo ×11 a ×15, perto do previsto por n log n.

Na tela:

- Gráfico de barras (escala log): tempo médio por tamanho, Merge e Quick
- Números grandes à direita: ×11 a ×15 medido · ×12 a ×17 previsto · ×100 seria n^2

FALA

Neste gráfico, em escala logarítmica, as barras sobem em degraus quase iguais. Cada vez que multiplicamos os dados por 10, o tempo cresceu entre 11 e 15 vezes. A fórmula n log n prevê entre 12 e 17 vezes, então a prática bateu com a teoria. Se o algoritmo fosse n ao quadrado, cada degrau seria de 100 vezes.

Transição: *Até aqui, os números eram aleatórios. O João Vitor mostra o que acontece com outras entradas.*

Slide 11 · Outros tipos de entrada (300.000 dados)

Quem fala: João Vitor · **Tempo:** 0:45 (de 10:45 a 11:30)
Mensagem principal: O Quicksort venceu nas 4 entradas; nas ordenadas, invertidas e repetidas, de 45% a 55% mais rápido.

Na tela:
- Gráfico de barras: Aleatória, Já ordenada, Invertida, Muitos repetidos
- Cartão roxo: 45% a 55% · tópico do pivô ideal · quadro “No dia a dia” (estante)

FALA

Também testamos 300 mil números em quatro tipos de entrada: aleatória, já ordenada, invertida e com muitos valores repetidos. O Quicksort venceu nas quatro, e nas três últimas foi de 45% a 55% mais rápido. Num vetor já ordenado, o elemento do meio é exatamente o pivô ideal, por isso o Quicksort não caiu no pior caso. É como arrumar uma estante que já está quase em ordem.

Transição: *Juntando tudo, cada algoritmo tem vantagens e desvantagens.*

Slide 12 · Vantagens e desvantagens

Quem fala: João Vitor · **Tempo:** 0:45 (de 11:30 a 12:15)
Mensagem principal: O Mergesort ganha em garantia; o Quicksort, em velocidade e memória.

Na tela:
- Dois cartões: MERGESORT (azul) e QUICKSORT (lilás), com as desvantagens em roxo

FALA

O Mergesort tem tempo garantido, n log n sempre, é estável e, na versão bottom-up, não usa recursão. A desvantagem é a memória extra e as cópias, e foi por isso que ele ficou mais lento no nosso teste. O Quicksort foi até 15% mais rápido e usa pouca memória, porque ordena no próprio vetor. As desvantagens são o pior caso n ao quadrado, se o pivô for ruim, e o fato de não ser estável.

Transição: *O Guilherme fecha com a nossa conclusão.*

Slide 13 · Qual algoritmo escolher?

Quem fala: Guilherme · **Tempo:** 0:45 (de 12:15 a 13:00)
Mensagem principal: Depende do objetivo: velocidade, Quicksort; garantia e estabilidade, Mergesort.

Na tela:

- 3 tópicos, “Curiosidade” sobre o pivô, “Obrigado!” e a placa de direção MERGESORT ← | → QUICKSORT

FALA

Então, qual algoritmo escolher? Depende do objetivo. Se o que importa é velocidade e pouca memória, o Quicksort, desde que o pivô seja bem escolhido. Se é preciso garantir o tempo no pior caso ou manter a ordem dos elementos iguais, o Mergesort. Tanto que a própria biblioteca do C++ oferece os dois. E uma curiosidade: um pivô mal escolhido transforma o Quicksort num algoritmo quadrático. Obrigado pela atenção. Ficamos à disposição para perguntas.

Transição: *(fim; abrir para perguntas)*

3. Roteiro da demonstração (Guilherme, 1:00, opcional)

Preparação (antes de o professor chamar o grupo): pasta do ZIP extraída no notebook que vai no projetor; Dev-C++ instalado; o painel já compilado uma vez (opção 7); o PAINEL_ADM.bat aberto e minimizado; a janela do console com fonte grande (botão direito na barra de título → Propriedades → Fonte → 20 ou mais).

Passo	Tempo	O que fazer	O que deve aparecer	O que dizer
D1	0:30	Alt+Tab para o painel. Digitar **2** e Enter.	"Executando 200 ordenações", 5 linhas de ####, a tabela de médias e o relatório abrindo no navegador.	"Com a opção 2, o painel roda as 200 ordenações em uns dois segundos e abre o relatório."
D2	0:30	Voltar ao painel, digitar **11** e Enter. Rolar até a Tabela 5.	Tabelas 1 a 5 (Algoritmo × Média, Mediana, Desvio, Variância, CV, Máx/Mín) e a Tabela 6 de crescimento.	"A opção 11 mostra a estatística no formato que o professor pediu. Os números desta máquina mudam, mas o padrão é o mesmo: empate nos pequenos e Quicksort na frente nos grandes."

Plano B

Falha	O que fazer (sem parar a apresentação)
O painel diz "Compilador NAO ENCONTRADO" ou não abre	Pular a demonstração: "os números estão nos próximos slides" e seguir para o slide 4.
O navegador não abre o relatório	Seguir direto para o passo D2.
Tela do console pequena ou cortada	Mostrar só o passo D1 e voltar para os slides.
Estourou o tempo	Pular a demonstração inteira (−1:00).

4. Checklist

Na véspera (12/11)

- ■ Extrair o ZIP no notebook da apresentação, abrir o PAINEL_ADM.bat e rodar as opções 7, 2 e 11 uma vez.
- ■ Abrir o slides.pptx no PowerPoint: conferir os 6 nomes no slide 1 e se os títulos aparecem na fonte Arial Narrow.
- ■ Ensaiar com cronômetro, na ordem e com as trocas de pessoa. Meta: 13:00. Limite: 15:00.
- ■ Cada um ler a sua fala em voz alta pelo menos três vezes. Não precisa decorar palavra por palavra: decorem a mensagem principal e os números.
- ■ Cada um ler as perguntas prováveis (seção 5) e responder em voz alta sem olhar.
- ■ Copiar o ZIP e o slides.pptx para um pendrive.

30 minutos antes

- ■ Ligar o notebook no projetor e desligar as notificações.
- ■ Abrir o slides.pptx no modo apresentação. As falas estão nas anotações do apresentador.
- ■ Abrir o PAINEL_ADM.bat, deixar minimizado e aumentar a fonte do console.
- ■ Combinar o sinal de tempo: quem estiver livre levanta a mão aos 10 e aos 13 minutos.

5. Perguntas prováveis e respostas

Quem responde é quem apresentou o assunto. Se a pessoa travar, o Guilherme assume.

1. Qual a diferença entre o Mergesort bottom-up e o top-down? (responde: Jeferson)
O top-down é recursivo: divide o vetor ao meio até sobrar 1 elemento e junta na volta da recursão. O bottom-up não divide: começa com blocos de 1 e junta em laços, dobrando o tamanho do bloco a cada passada. O custo é o mesmo, n log n, mas o bottom-up não usa recursão.

2. Por que o Mergesort é n log n? (responde: Jeferson)
Porque o bloco dobra a cada passada, então são log de n passadas (19 para 300 mil), e cada passada percorre os n elementos uma vez.

3. Se os dois são n log n, por que o Quicksort foi mais rápido? (responde: Juan)
A notação O esconde as constantes. O Mergesort copia todos os elementos para o vetor auxiliar e de volta a cada passada; o Quicksort só troca elementos no próprio vetor e aproveita melhor a memória cache.

4. Quando o Quicksort vira n²? Como vocês evitaram? (responde: Gustavo)
Quando o pivô é sempre o menor ou o maior elemento, por exemplo, o primeiro elemento num vetor já ordenado. Usamos o elemento do meio, que num vetor ordenado é o pivô ideal: 300 mil números já ordenados levaram 3,1 ms.

5. O que é um algoritmo estável? (responde: Jeferson)
É o que mantém a ordem original dos elementos iguais, como a fila do banco: quem chegou antes continua na frente. O Mergesort é estável (no empate, pega o da esquerda); o Quicksort não.

6. Como mediram os vetores pequenos, que ordenam em menos de 1 microssegundo? (responde: Guilherme)
Cada amostra ordena k vetores diferentes (k = 20.000 ÷ n) e divide o tempo por k. Numa versão preliminar repetíamos o mesmo vetor, e os tempos saíram baixos demais, porque o processador “decora” a entrada repetida; por isso usamos vetores diferentes.

7. Por que 20 amostras e semente fixa? (responde: Guilherme)
As 20 amostras são do enunciado e permitem calcular a estatística. A semente fixa (42) faz o gerador produzir sempre os mesmos números, então qualquer pessoa consegue repetir o experimento.

8. O que é o coeficiente de variação? (responde: Juan)
É o desvio padrão dividido pela média, em porcentagem. Mostra o quanto os tempos variam em relação à média: ficou entre 1,5% e 12,5%, de baixo a moderado.

9. Como se lê o boxplot? (responde: Igor)
A caixa vai do primeiro ao terceiro quartil (a metade do meio das amostras), a linha interna é a mediana e os pontos soltos são valores fora do padrão. Caixas que se sobrepõem indicam empate.

10. Por que as medições do artigo foram num servidor Linux? (responde: Guilherme)
Usamos um servidor Linux de terceiros, acessado remotamente, para ter um ambiente estável e igual para todas as medições. O mesmo programa roda no Windows pelo painel, e os números mudam de máquina para máquina, mas o padrão se mantém, como mostramos na demonstração.

11. Por que não fizeram o Quicksort personalizado? (responde: Guilherme)
Porque o item 3 do enunciado, que pedia o pivô personalizado, foi retirado. Comparamos o algoritmo do grupo com o Quicksort clássico, como pede o restante do enunciado.

12. Quanta memória o Mergesort usa a mais? (responde: Jeferson)
Um vetor auxiliar do mesmo tamanho: para 300 mil inteiros de 4 bytes, cerca de 1,2 MB.